

II.2415 Advanced Algorithms and Programming

Lab 4: Linear Data Structures: Team T24

Members:

YADAV Anshuman Krishna (exercise 1)

MAHALINGAM Nithees (exercise 2)

SARAVANAN Arun Prasath (exercise 3)

Github Link:

<https://github.com/anshuman-krishna/advanced-algorithms-and-programming>

Exercise 1: YADAV Anshuman Krishna

Recursive Comment Thread Traversal

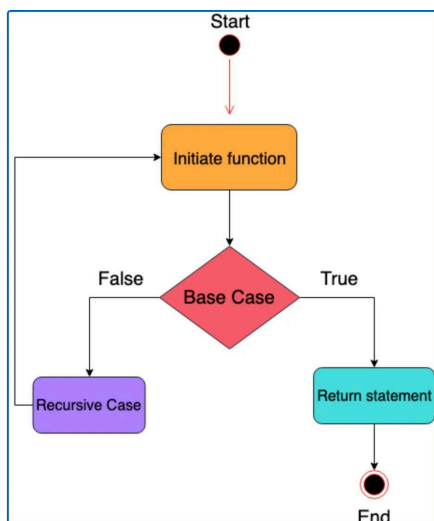
1. Base Structure

```
STRUCTURE CommentNode
comment_id : String
user_id : String
content : String
timestamp : Integer
likes : Integer
replies : List CommentNode
END STRUCTURE
```

Time Complexity : $O(1)$

Space Complexity : $O(1)$

2. Recursive Traversal



A. display_thread

...

to be included in T24_LAB4_BFM

...

B. count_total_comments

```
FUNCTION count_total_comments(comment) -> Integer
total <- 1
FOR EACH reply IN comment.replies DO
total <- total + count_total_comments(reply)
END FOR
RETURN total
```

Time Complexity : $O(1)$

Space Complexity : $O(d)$, d is max depth

C. total_likes

```
FUNCTION total_likes(comment) -> Integer
sum_likes <- comment.likes

FOR EACH reply IN comment.replies DO
sum_likes <- sum_likes + total_likes(reply)
END FOR
RETURN sum_likes
```

Time Complexity : $O(n)$, since we visit every node

Space Complexity : $O(d)$, recursive call stack

D. find_deepest_reply

```
FUNCTION find_deepest_reply(comment) -> Integer
max_depth <- 0

FOR EACH reply IN comment.replies DO
depth <- find_deepest_reply(reply)
IF depth > max_depth THEN
max_depth <- depth
END IF
END FOR
RETURN max_depth + 1
```

Time Complexity : $O(n)$, since we visit every node

Space Complexity : $O(d)$, recursive call stack

3. Implement Search Functionality

A. search_by_user

...

to be included in T24_LAB4_BFM

...

B. contains_keyword

FUNCTION contains_keyword(keyword, comment) -> Boolean

IF comment.content.Contains(keyword) THEN

RETURN TRUE

END IF

FOR EACH reply IN comment.replies DO

IF contains_keyword(keyword, reply) = TRUE THEN

RETURN TRUE

END IF END FOR

RETURN FALSE

Time Complexity : $O(n)$, worst case,

Space Complexity : $O(d)$, recursive call stack

4. Implement Deletion with Cascade

...

to be included in T24_LAB4_BFM

...

Complexity Analysis Questions:

1. What is the time complexity of traversing an entire comment thread with n-total comments?

Answer:

The time complexity will be $O(n)$ because we are dealing with a tree-like structure, hence we will be visiting each single node once to perform various operations (like counting, displaying, etc.)

II.2415 Advanced Algorithms and Programming

Lab 4: Linear Data Structures: Team T24

2. How does recursion depth relate to the comment nesting structure? What happens with very deep threads (1000+ levels?)

Answer:

The recursion depth is equal to the thread's nesting level. Every time someone replies to a pre-existing reply, the function calls itself one level deeper. For a thread with 1000+ levels, it will open 1000+ nested functions at the same time, which can exceed a machine's built-in memory limits and can cause a stack-overflow crash.

3. Compare memory usage between recursive and iterative stack-based implementations?

Answer:

Both the implementations would have a space complexity of $O(d)$, mathematically, where d will be the depth of the thread.

...

memory usage // research before BFM

...

what we already knew,

If we're building a platform like Reddit or X where threads can go many levels deep, iterative is a better choice to avoid crashes.

why, though?

// research before BFM

Exercise 2: MAHALINGAM Nithees

Recursive Content Aggregation with Divide & Conquer

1. Post Structure

Structure Post

post_id

user_id

content_preview

time_stamp

```
likes
comments
shares
engagement_score
End Structure
```

2. Divide & Conquer

Part A : Maximum Engagement

```
Function max_engagement(posts, left, right)
  If left = right
    return posts[left]
  mid <- (left + right) / 2
  left_max <- max_engagement(posts, left, mid)
  right_max <- max_engagement(posts, mid + 1, right)
  If left_max.engagement_score > right_max.engagement_score
    return left_max
  Else
    return right_max
End
```

Part B : Total and Average

Total

```
Function sum_engagement(posts, left, right)
  If left = right
    return posts[left].engagement_score
  mid <- (left + right) / 2
  left_sum <- sum_engagement(posts, left, mid)
  right_sum <- sum_engagement(posts, mid + 1, right)
  return left_sum + right_sum
```

End

Sum

```
Function average_engagement(posts, left, right)
  total <- sum_engagement(posts, left, right)
  n <- right - left + 1
  return total / n
```

End

Part C : Count by Threshold

.... // BFM

Part D : Merge Sort by Engagement

Merge Function

```
Function merge(posts, left, mid, right)
  Create arrays L and R
  Copy posts[left..mid] into L
  Copy posts[mid+1..right] into R
  i <- 0, j <- 0, k <- left
  While i < length(L) AND j < length(R)
    If L[i].engagement_score ≤ R[j].engagement_score
      posts[k] <- L[i]
      i <- i + 1
    Else
      posts[k] <- R[j]
      j <- j + 1
    k <- k + 1
  Copy remaining elements of L
  Copy remaining elements of R
End
```

Merge Sort

```
Function merge_sort(posts, left, right)
  If left < right
    mid <- (left + right) / 2
    merge_sort(posts, left, mid)
    merge_sort(posts, mid + 1, right)
    merge(posts, left, mid, right)
End
```

3. Peak Hours Analysis

..///BFM

Complexity Analysis Questions:

1. What is the time complexity of recursive max_engagement? Prove it.

Answer:

II.2415 Advanced Algorithms and Programming

Lab 4: Linear Data Structures: Team T24

Recurrence: $T(n) = 2T(n/2) + O(1)$

Using Master Theorem: $T(n) = O(n)$

The array is divided into two halves and each element is compared once.

2. Compare recursive merge sort ($O(n \log n)$) with iterative insertion sort ($O(n^2)$) for sorting 10,000 posts.

Answer: ..//BFM

3. What is the recursion depth for merge sort on n elements?

Answer:

$O(\log n)$

Array is divided into halves repeatedly until size = 1.

4. For `find_peak_hour`, why can we use binary search-style recursion? What property must the array satisfy?

Answer:

...//BFM

Exercise 3: SARAVANAN Arun Prasath

Converting Recursion to Iteration with Explicit Stack

1. Recursive Flatten

```
FUNCTION flatten_recursive(comment):
  result = [comment]
  FOR each reply IN comment.replies:
    result = result + flatten_recursive(reply)
  RETURN result
```

2. Iterative Flatten (Using Stack)

II.2415 Advanced Algorithms and Programming

Lab 4: Linear Data Structures: Team T24

```
FUNCTION flatten_iterative(comment):
stack = [(comment, "START")]
result = []
while stack is not empty:
node, state = POP(stack)
if state == "START":
result.append(node)
stack.push((node, "DONE"))
for reply in reversed(node.replies):
stack.push((reply, "START"))
return result
```

3. Tail Recursive Counting

```
FUNCTION count_comments_tail(comment, count):
count = count + 1
for reply in comment.replies:
count = count_comments_tail(reply, count)
return count
```

4. Iterative Counting (Loop Version)

```
FUNCTION count_comments_loop(comment):
stack = [comment]
count = 0
while stack is not empty:
node = POP(stack)
count = count + 1
for reply in node.replies:
stack.append(reply)
return count
```

Complexity Analysis Questions:

1. For a thread with depth d and total n comments, compare maximum stack size needed for:
 - Recursive implementation
 - Iterative with explicit stack

II.2415 Advanced Algorithms and Programming

Lab 4: Linear Data Structures: Team T24

Answer:

2. When would you prefer recursive version? When iterative?

Answer:

3. How does tail recursion conversion reduce stack usage? Can all recursive functions be made tail recursive?

Answer:

4. Explain the state machine approach—why is it necessary for functions that process nested structures?

Answer:

5. In a production social network with user-generated nested comments, which approach would you choose and why

...

to be continued in T24_LAB4_BFM

thanks.